



Animations

An animation that illustrates deadlock is available at the Premium Web site. The reader is encouraged to view the animation to reinforce concepts from this chapter.

6.14 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

banker's algorithm circular wait consumable resource deadlock deadlock avoidance deadlock detection	deadlock prevention hold and wait joint progress diagram memory barrier message mutual exclusion	pipe preemption resource allocation graph reusable resource spinlock starvation
--	---	--

Review Questions

- 6.1. Give examples of reusable and consumable resources.
- 6.2. What are the three conditions that must be present for deadlock to be possible?
- 6.3. What are the four conditions that create deadlock?
- 6.4. How can the hold-and-wait condition be prevented?
- 6.5. List two ways in which the no-preemption condition can be prevented.
- 6.6. How can the circular-wait condition be prevented?
- 6.7. What is the difference among deadlock avoidance, detection, and prevention?

Problems

- 6.1. Show that the four conditions of deadlock apply to Figure 6.1a.
- 6.2. Show how each of the techniques of prevention, avoidance, and detection can be applied to Figure 6.1.
- 6.3. For Figure 6.3, provide a narrative description of each of the six depicted paths, similar to the description of the paths of Figure 6.2 provided in Section 6.1.
- 6.4. It was stated that deadlock cannot occur for the situation reflected in Figure 6.3. Justify that statement.
- 6.5. Given the following state for the Banker's Algorithm.
6 processes P0 through P5
4 resource types: A (15 instances); B (6 instances)
C (9 instances); D (10 instances)
Snapshot at time T0:

Available			
A	B	C	D
6	3	5	4

Process	Current allocation				Maximum demand			
	A	B	C	D	A	B	C	D
P0	2	0	2	1	9	5	5	5
P1	0	1	1	1	2	2	3	3
P2	4	1	0	2	7	5	4	4
P3	1	0	0	1	3	3	3	2
P4	1	1	0	0	5	2	2	1
P5	1	0	1	1	4	4	4	4

- a. Verify that the Available array has been calculated correctly.
 - b. Calculate the Need matrix.
 - c. Show that the current state is safe, that is, show a safe sequence of processes. In addition, to the sequence show how the Available (working array) changes as each process terminates.
 - d. Given the request (3,2,3,3) from Process P5. Should this request be granted? Why or why not?
- 6.6. In the code below, three processes are competing for six resources labeled A to F.
- a. Using a resource allocation graph (Figures 6.5 and 6.6), show the possibility of a deadlock in this implementation.
 - b. Modify the order of some of the get requests to prevent the possibility of any deadlock. You cannot move requests across procedures, only change the order inside each procedure. Use a resource allocation graph to justify your answer.

<pre> void P0() { while (true) { get(A); get(B); get(C); // critical region: // use A, B, C release(A); release(B); release(C); } } </pre>	<pre> void P1() { while (true) { get(D); get(E); get(B); // critical region: // use D, E, B release(D); release(E); release(B); } } </pre>	<pre> void P2() { while (true) { get(C); get(F); get(D); // critical region: // use C, F, D release(C); release(F); release(D); } } </pre>
--	--	--

- 6.7. A spooling system (Figure 6.17) consists of an input process I, a user process P, and an output process O connected by two buffers. The processes exchange data in blocks of equal size. These blocks are buffered on a disk using a floating boundary between the

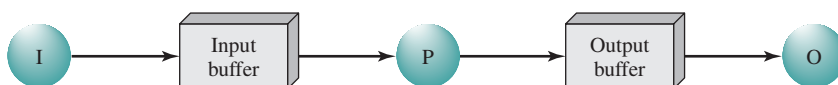


Figure 6.17 A Spooling System

input and the output buffers, depending on the speed of the processes. The communication primitives used ensure that the following resource constraint is satisfied:

$$i + o \leq \max$$

where

- max = maximum number of blocks on disk
- i* = number of input blocks on disk
- o* = number of output blocks on disk

The following is known about the processes:

1. As long as the environment supplies data, process I will eventually input it to the disk (provided disk space becomes available).
2. As long as input is available on the disk, process P will eventually consume it and output a finite amount of data on the disk for each block input (provided disk space becomes available).
3. As long as output is available on the disk, process O will eventually consume it.

Show that this system can become deadlocked.

- 6.8. Suggest an additional resource constraint that will prevent the deadlock in Problem 6.7 but still permit the boundary between input and output buffers to vary in accordance with the present needs of the processes.
- 6.9. In the THE multiprogramming system [DIJK68], a drum (precursor to the disk for secondary storage) is divided into input buffers, processing areas, and output buffers, with floating boundaries, depending on the speed of the processes involved. The current state of the drum can be characterized by the following parameters:

- max = maximum number of pages on drum
- i* = number of input pages on drum
- p* = number of processing pages on drum
- o* = number of output pages on drum
- reso = minimum number of pages reserved for output
- resp = minimum number of pages reserved for processing

Formulate the necessary resource constraints that guarantee that the drum capacity is not exceeded and that a minimum number of pages is reserved permanently for output and processing.

- 6.10. In the THE multiprogramming system, a page can make the following state transitions:
1. empty → input buffer (input production)
 2. input buffer → processing area (input consumption)
 3. processing area → output buffer (output production)
 4. output buffer → empty (output consumption)
 5. empty → processing area (procedure call)
 6. processing area → empty (procedure return)
- a. Define the effect of these transitions in terms of the quantities *i*, *o*, and *p*.
- b. Can any of them lead to a deadlock if the assumptions made in Problem 6.6 about input processes, user processes, and output processes hold?
- 6.11. Consider a system with a total of 150 units of memory, allocated to three processes as shown:

Process	Max	Hold
1	70	45
2	60	40
3	60	15

Apply the banker's algorithm to determine whether it would be safe to grant each of the following requests. If yes, indicate a sequence of terminations that could be guaranteed possible. If no, show the reduction of the resulting allocation table.

- a. A fourth process arrives, with a maximum memory need of 60 and an initial need of 25 units.
 - b. A fourth process arrives, with a maximum memory need of 60 and an initial need of 35 units.
- 6.12.** Evaluate the banker's algorithm for its usefulness in an OS.
- 6.13.** A pipeline algorithm is implemented so that a stream of data elements of type T produced by a process P_0 passes through a sequence of processes $P_1, P_2, \dots, P_{n-1}$, which operates on the elements in that order.
- a. Define a generalized message buffer that contains all the partially consumed data elements and write an algorithm for process P_i ($0 \leq i \leq n - 1$), of the form

repeat

 receive from predecessor;
 consume element;
 send to successor;

forever

Assume P_0 receives input elements sent by P_{n-1} . The algorithm should enable the processes to operate directly on messages stored in the buffer so that copying is unnecessary.

- b. Show that the processes cannot be deadlocked with respect to the common buffer.
- 6.14.** Suppose the following two processes, foo and bar, are executed concurrently and share the semaphore variables S and R (each initialized to 1) and the integer variable x (initialized to 0).

<pre>void foo() { do { semWait(S); semWait(R); x++; semSignal(S); SemSignal(R); } while (1); }</pre>	<pre>void bar() { do { semWait(R); semWait(S); x--; semSignal(S); SemSignal(R); } while (1); }</pre>
---	---

- a. Can the concurrent execution of these two processes result in one or both being blocked forever? If yes, give an execution sequence in which one or both are blocked forever.
 - b. Can the concurrent execution of these two processes result in the indefinite postponement of one of them? If yes, give an execution sequence in which one is indefinitely postponed.
- 6.15.** Consider a system consisting of four processes and a single resource. The current state of the claim and allocation matrices are:

$$\mathbf{C} = \begin{pmatrix} 3 \\ 2 \\ 9 \\ 7 \end{pmatrix} \quad \mathbf{A} = \begin{pmatrix} 1 \\ 1 \\ 3 \\ 2 \end{pmatrix}$$

What is the minimum number of units of the resource needed to be available for this state to be safe?

- 6.16.** Consider the following ways of handling deadlock: (1) banker's algorithm, (2) detect deadlock and kill thread, releasing all resources, (3) reserve all resources in advance, (4) restart thread and release all resources if thread needs to wait, (5) resource ordering, and (6) detect deadlock and roll back thread's actions.
- One criterion to use in evaluating different approaches to deadlock is which approach permits the greatest concurrency. In other words, which approach allows the most threads to make progress without waiting when there is no deadlock? Give a rank order from 1 to 6 for each of the ways of handling deadlock just listed, where 1 allows the greatest degree of concurrency. Comment on your ordering.
 - Another criterion is efficiency; in other words, which requires the least processor overhead. Rank order the approaches from 1 to 6, with 1 being the most efficient, assuming that deadlock is a very rare event. Comment on your ordering. Does your ordering change if deadlocks occur frequently?
- 6.17.** Comment on the following solution to the dining philosophers problem. A hungry philosopher first picks up his left fork; if his right fork is also available, he picks up his right fork and starts eating; otherwise he puts down his left fork again and repeats the cycle.
- 6.18.** Suppose that there are two types of philosophers. One type always picks up his left fork first (a "lefty"), and the other type always picks up his right fork first (a "righty"). The behavior of a lefty is defined in Figure 6.12. The behavior of a righty is as follows:

```

begin
  repeat
    think;
    wait ( fork[ (i+1) mod 5] );
    wait ( fork[i] );
    eat;
    signal ( fork[i] );
    signal ( fork[ (i+1) mod 5] );
  forever
end;

```

Prove the following:

- Any seating arrangement of lefties and righties with at least one of each avoids deadlock.
 - Any seating arrangement of lefties and righties with at least one of each prevents starvation.
- 6.19.** Figure 6.18 shows another solution to the dining philosophers problem using monitors. Compare to Figure 6.14 and report your conclusions.
- 6.20.** In Table 6.3, some of the Linux atomic operations do not involve two accesses to a variable, such as `atomic_read(atomic_t *v)`. A simple read operation is obviously atomic in any architecture. Therefore, why is this operation added to the repertoire of atomic operations?
- 6.21.** Consider the following fragment of code on a Linux system.

```

read_lock(&mr_rwlock);
write_lock(&mr_rwlock);

```

Where `mr_rwlock` is a reader-writer lock. What is the effect of this code?

```

monitor dining_controller;
enum states {thinking, hungry, eating} state[5];
cond needFork[5]                                /* condition variable */

void get_forks(int pid)                          /* pid is the philosopher id number */
{
    state[pid] = hungry;                          /* announce that I'm hungry */
    if (state[(pid+1) % 5] == eating || (state[(pid-1) % 5] == eating)
        cwait(needFork[pid]);                      /* wait if either neighbor is eating */
    state[pid] = eating;                          /* proceed if neither neighbor is eating */
}

void release_forks(int pid)
{
    state[pid] = thinking;
    /* give right (higher) neighbor a chance to eat */
    if (state[(pid+1) % 5] == hungry) && (state[(pid+2) % 5] != eating)
        csignal(needFork[pid+1]);
    /* give left (lower) neighbor a chance to eat */
    else if (state[(pid-1) % 5] == hungry) && (state[(pid-2) % 5] != eating)
        csignal(needFork[pid-1]);
}

void philosopher[k=0 to 4]                      /* the five philosopher clients */
{
    while (true) {
        <think>;
        get_forks(k);                             /* client requests two forks via monitor */
        <eat spaghetti>;
        release_forks(k);                         /* client releases forks via the monitor */
    }
}

```



Figure 6.18 Another Solution to the Dining Philosophers Problem Using a Monitor

- 6.22.** The two variables *a* and *b* have initial values of 1 and 2, respectively. The following code is for a Linux system:

Thread 1	Thread 2
<i>a</i> = 3;	—
mb();	—
<i>b</i> = 4;	<i>c</i> = <i>b</i> ;
—	rmb();
—	<i>d</i> = <i>a</i> ;

What possible errors are avoided by the use of the memory barriers?